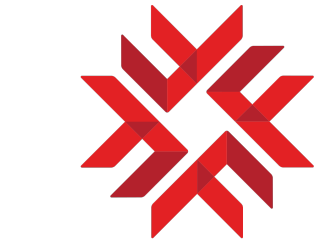
Tensorflow II

One-Stage Object Detectors

Fatemeh Dehrouyeh



Contents

1. One-Stage vs. Two-Stage Detectors
2. Feature Maps and Prediction Locations
3. Anchor Boxes
4. One-Stage Anchor-Based Object Detector — SSD Example
   - Lab
5. One-Stage Anchor-Free Object Detector — YOLO Example
   - Original YOLOv1
   - Lab

Current Section

1 One-Stage vs. Two-Stage Detectors

2 Feature Maps and Prediction Locations

3 Anchor Boxes

4 One-Stage Anchor-Based Object Detector — SSD Example

5 One-Stage Anchor-Free Object Detector — YOLO Example

Object Detection Architectures

Modern object detectors are commonly grouped into two families:

- **One-stage detectors**
- **Two-stage detectors**

Main Question

How does the model produce object classes and bounding boxes from an image?

Two-Stage Detectors

Two-stage detectors solve object detection in two main stages.

Stage 1: Generate candidate object regions

Stage 2: Classify and refine those regions

Examples include:

- R-CNN
- Fast R-CNN
- Faster R-CNN

Intuition

First ask: *Where might objects be?*
Then ask: *What object is in each region, and where exactly is it?*

One-Stage Detectors

One-stage detectors predict object locations and class scores directly from image features without a separate region-proposal stage.

$$\boxed{\text{Image} \rightarrow \text{Feature Extractor} \rightarrow \text{Boxes + Classes}}$$

Examples include:

- YOLO
- SSD
- RetinaNet

Intuition

Instead of first proposing regions and then classifying them, one-stage detectors make detection predictions directly.

One-Stage vs. Two-Stage Detection

Aspect	One-Stage	Two-Stage
Main idea	Direct detection	Region proposals first
Detection stages	One	Two
Region proposal step	No separate step	Yes
Speed	Usually faster	Usually slower
Complexity	Simpler inference	More complex
Examples	YOLO, SSD	Faster R-CNN

Practical View

One-stage detectors are commonly used when fast detection is important, while two-stage detectors separate region proposal from object classification and localization.

Current Section

1 One-Stage vs. Two-Stage Detectors

2 Feature Maps and Prediction Locations

3 Anchor Boxes

4 One-Stage Anchor-Based Object Detector — SSD Example

5 One-Stage Anchor-Free Object Detector — YOLO Example

Where Are Detection Predictions Made?

You already know that a CNN converts an image into feature maps.
In object detection, these feature maps also provide the spatial locations where predictions are made.

For example:

$$640 \times 640 \text{ image} \rightarrow 80 \times 80 \text{ feature map}$$

The 80×80 feature map contains:

$$80 \times 80 = 6400$$

possible prediction locations.

Key Idea

The detection grid is not randomly created.
It comes from the spatial structure of the feature map.

Prediction Locations and Scale

The spacing between prediction locations depends on the feature-map stride.
For example:

$$\frac{640 \text{ (image size)}}{80 \text{ (feature map size)}} = 8 \text{ pixels (effective stride)}$$

so an 80×80 feature map has an effective stride of approximately 8 pixels.

Detectors may use several feature maps:

$$80 \times 80, \qquad 40 \times 40, \qquad 20 \times 20$$

with approximate strides: $\quad 8, \qquad 16, \qquad 32$

- finer feature maps help detect smaller objects,
- coarser feature maps help detect larger objects.

Next Question

At each prediction location, how does the detector represent possible bounding boxes?

Anchor Boxes

Current Section

1 One-Stage vs. Two-Stage Detectors

2 Feature Maps and Prediction Locations

3 Anchor Boxes

4 One-Stage Anchor-Based Object Detector — SSD Example

5 One-Stage Anchor-Free Object Detector — YOLO Example

Anchor Boxes Summary

Anchor boxes are predefined boxes placed at fixed grid locations across a feature map. Their centers are usually aligned with the centers of grid cells, and each location may contain several anchors with different sizes and aspect ratios. The detector is trained on labeled images containing a predefined set of object classes. During training, it compares anchors with the true bounding boxes using measures such as Intersection over Union, or IoU, and learns which image features indicate an object. For each anchor, it learns to predict whether an object is present, which class it belongs to, and how the anchor should be shifted or resized to better fit the object. After training, the same anchor layout is applied to new images, and the trained detector uses the learned features to locate and classify objects.

What Is an Anchor Box?

An **anchor box** is a predefined reference box associated with a location on a feature map.

Instead of predicting a bounding box completely from scratch, an anchor-based detector predicts how an anchor should be adjusted:

$$\boxed{\text{Anchor Box} + \text{Predicted Adjustment} \rightarrow \text{Predicted Bounding Box}}$$

Key Idea

Anchor boxes provide predefined starting locations, sizes, and shapes from which the detector can predict object boxes.

Where Are Anchor Boxes Placed?

Anchor boxes are associated with spatial locations on a feature map.

For example, an:

$$80 \times 80$$

feature map contains:

$$80 \times 80 = 6400$$

spatial locations.

At each location, the detector may define several anchor boxes.

$$\boxed{\text{Feature-Map Location} \rightarrow \text{Several Anchors}}$$

Important

The anchor locations are not randomly selected.
They follow the spatial grid of the feature map.

Why Multiple Anchor Boxes?

Objects can have very different shapes.

For example:

$$\text{person} \rightarrow \text{tall and narrow}$$

$$\text{car} \rightarrow \text{wide and short}$$

$$\text{traffic sign} \rightarrow \text{approximately square}$$

Therefore, detectors may place several anchors at the same location with different:

- sizes,
- widths and heights,
- aspect ratios.

Goal

Provide starting boxes that can approximately match objects with different sizes and shapes.

Anchor Box Aspect Ratio

The **aspect ratio** describes the relationship between box width and height:

$$\text{Aspect Ratio} = \frac{\text{Width}}{\text{Height}}$$

For example:

$$1:1 \qquad 2:1 \qquad 1:2$$

can represent:

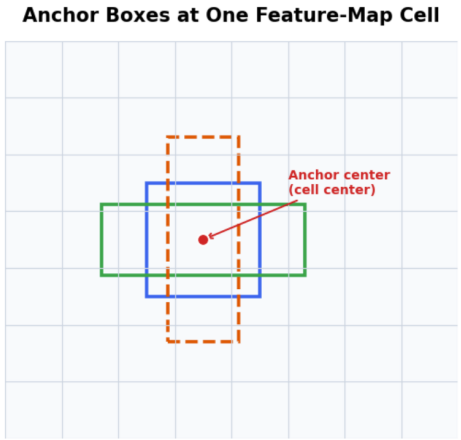
square wide tall

Key Idea

Different anchor shapes allow the detector to better match different types of objects.

Understanding Anchor Boxes

Anchor boxes are predefined rectangles of different sizes and aspect ratios, centered at feature-map grid locations.



All three anchors share the same center but have different widths and heights.

For example, an 80×80 feature map with 3 anchors per location produces:

$$80 \times 80 \times 3 = 19,200 \text{ anchors}$$

Key Idea

These are initial candidate boxes, not 19,200 detected objects.
The model learns which anchors are likely to contain objects and how to adjust their positions and dimensions.

Feature Maps During One-Stage Detection

During Training:

1. The CNN applies trainable convolutional filters to input images and generates feature maps.
2. Classification and bounding-box regression heads use these feature maps to predict object classes and locations.
3. Predictions are compared with ground-truth labels and boxes to calculate the training losses.
4. Backpropagation updates the CNN and detection head weights.

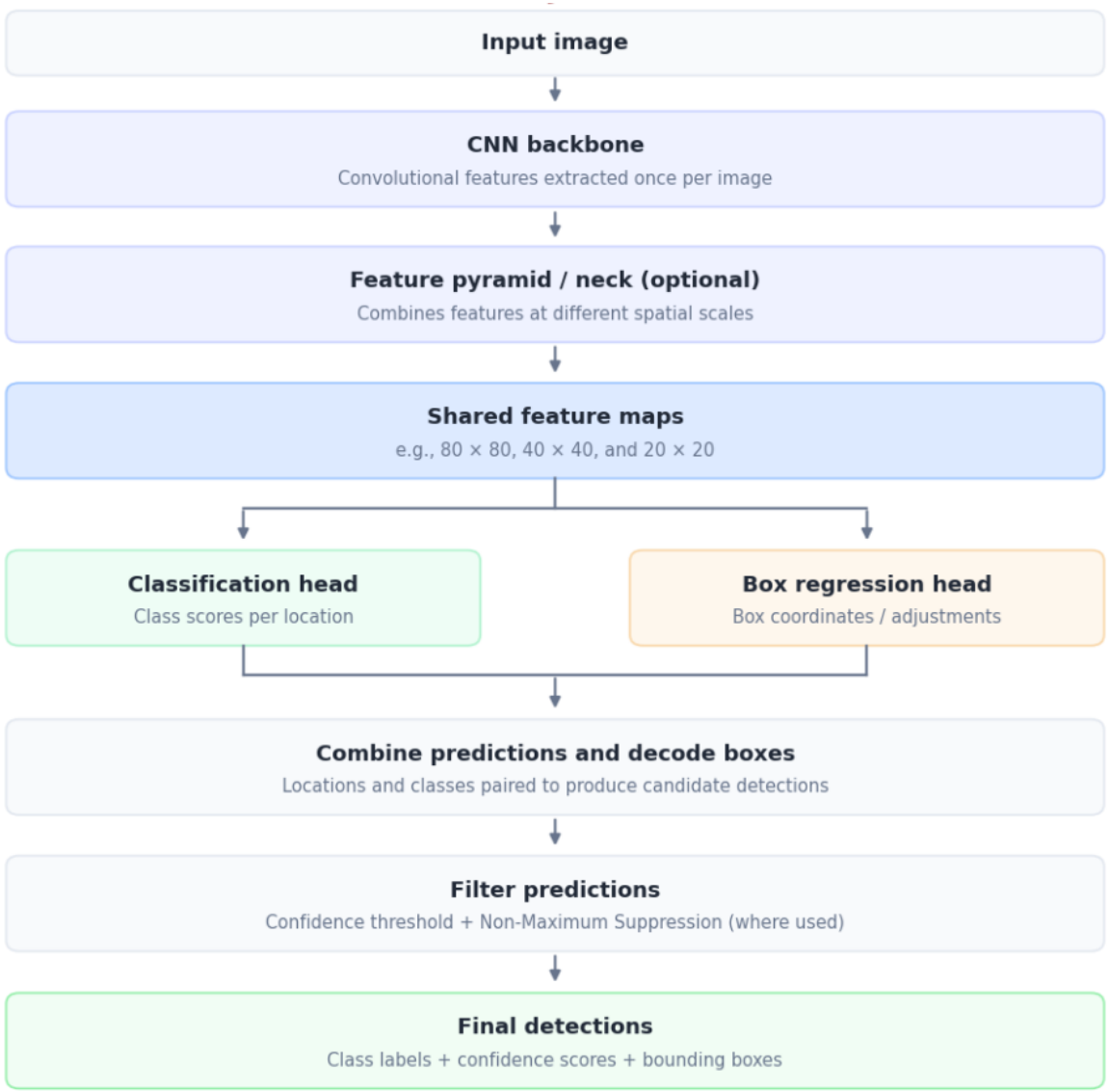
During Inference:

- Learned weights remain fixed.
- Each new image produces different feature maps.
- Detection heads use these maps to predict object classes and bounding boxes directly.

Key Concept

Feature maps are intermediate representations, not final predictions. Their values change with each input image, while the learned weights remain fixed during inference.

One-Stage Object Detector



How Are Anchors Assigned During Training?

During training, the detector knows the ground-truth bounding boxes.
Each anchor is compared with the ground-truth boxes using IoU.

$$\text{IoU} = \frac{\text{Intersection Area}}{\text{Union Area}}$$

For each anchor, the training procedure determines whether it should be:

- **positive**: matched to an object,
- **negative**: treated as background,
- **ignored**: not used for that part of the loss.

Important

The exact IoU thresholds depend on the detector and its training configuration.

What Does a Positive Anchor Learn?

During training, anchors are compared with ground-truth bounding boxes using Intersection over Union (IoU).
An anchor that sufficiently matches a ground-truth object is assigned as a **positive anchor**.

The detector learns to predict:

1. **Objectness:** Whether the anchor contains an object.
2. **Class:** Which object class is present.
3. **Position:** How the anchor should shift horizontally and vertically.
4. **Size:** How its width and height should change.

Conceptually:

$$\boxed{\text{Anchor} \rightarrow \text{Object/Class Prediction} + \text{Box Adjustment}}$$

Important

Different detectors organize these predictions differently. Some predict objectness and class scores separately, while others predict class-specific scores directly.

Bounding-Box Regression

An anchor rarely matches the object perfectly.
The network therefore predicts offsets that modify the anchor.

If the anchor is described by:

$$(x_a, y_a, w_a, h_a)$$

the network predicts adjustments such as:

$$(\Delta x, \Delta y, \Delta w, \Delta h)$$

These adjustments are used to produce the predicted box:

$$\boxed{\text{Anchor} + (\Delta x, \Delta y, \Delta w, \Delta h) \rightarrow \text{Refined Box}}$$

What the Model Learns

Move the box horizontally and vertically, and adjust its width and height, so that it better matches the ground-truth object.

What Happens to Anchors During Inference?

For a new image, the detector creates the same configured anchor pattern for the corresponding feature maps.

For each anchor, the trained network predicts information such as:

- object or class score,
- bounding-box adjustments.

The process is:

$$\text{Anchors} \rightarrow \text{Predicted Scores + Box Adjustments} \rightarrow \text{Candidate Detections}$$

Then:

$$\text{Confidence Filtering} \rightarrow \text{NMS} \rightarrow \text{Final Detections}$$

Important

Most anchors do not become final detections.

Anchor-Based vs. Anchor-Free Detectors

Anchor-Based

Start from predefined reference boxes.

- anchors are placed at prediction locations,
- the network scores them,
- the network predicts box adjustments.

Examples:

- SSD
- RetinaNet
- Faster R-CNN
- several older YOLO versions

Anchor-Free

Do not begin with predefined anchor shapes.
A detector may instead predict:

- object center locations,
- distances to box edges,
- box coordinates directly.

Examples:

- FCOS
- CenterNet
- YOLOX
- YOLOv8

Examples of Object Detection Architectures

Object detection models can be grouped according to their **detection strategy** and whether they use **anchor boxes**.

Two-Stage / Region-Based

- **R-CNN**
- **Fast R-CNN**
- **Faster R-CNN**
- **Mask R-CNN**

R-CNN and Fast R-CNN use external region proposals.
Faster R-CNN and Mask R-CNN traditionally use anchor boxes in the RPN.

One-Stage Anchor-Based

- **SSD**
- **RetinaNet**
- **YOLOv2**
- **YOLOv3**
- **YOLOv4**
- **YOLOv5**

Use predefined anchor boxes and predict classes and bounding-box adjustments directly.

One-Stage Anchor-Free

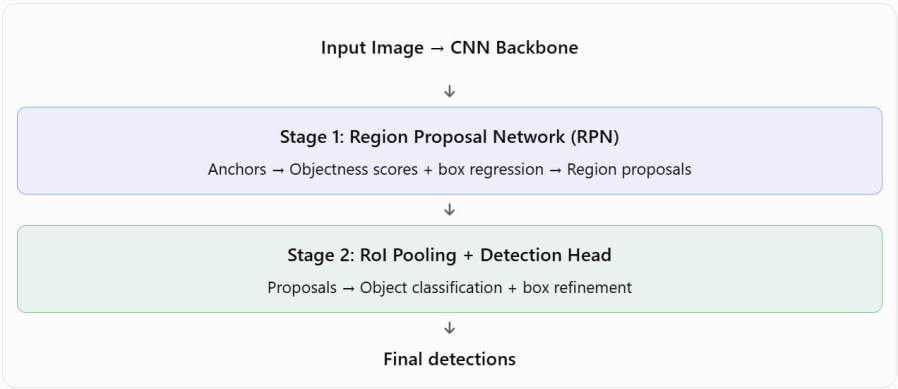
- **FCOS**
- **CenterNet**
- **YOLOX**
- **YOLOv8**

Predict object locations without predefined anchor-box shapes.

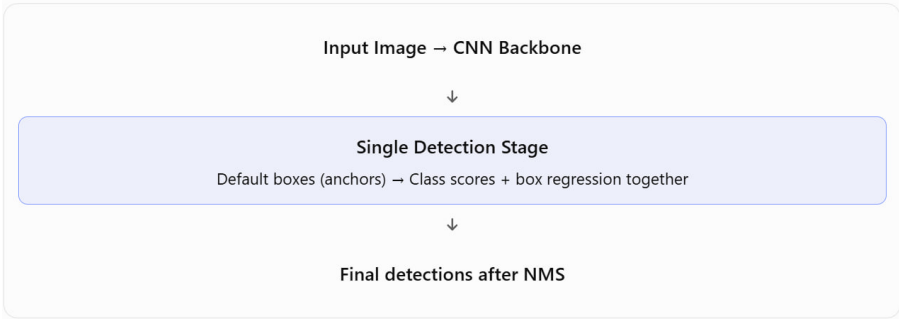
Key distinction: Two-stage vs. one-stage describes *how detection is performed*; anchor-based vs. anchor-free describes *how bounding boxes are generated.*

One-Stage vs Two-Stage Object Detection

Faster R-CNN: Two-Stage



SSD: One-Stage



Key Difference

Two-stage: Generates region proposals first, then classifies them.

One-stage: Predicts classes and bounding boxes directly.

Both can use anchor boxes. In Faster R-CNN, the second prediction depends on the output of the first prediction. In SSD, there is no second prediction that operates on the first prediction's boxes.

Current Section

1 One-Stage vs. Two-Stage Detectors

2 Feature Maps and Prediction Locations

3 Anchor Boxes

4 One-Stage Anchor-Based Object Detector — SSD Example
- Lab

5 One-Stage Anchor-Free Object Detector — YOLO Example

From Anchor Boxes to One-Stage Detectors

We now understand how an anchor-based detector can use feature-map locations to generate candidate bounding boxes.

The general idea is:

Feature Maps $\rightarrow$ Anchor Boxes $\rightarrow$ Scores + Box Adjustments $\rightarrow$ Candidate Detections

We have already seen anchor boxes used in the **Region Proposal Network of Faster R-CNN**.

Next, we will see how they are used in a one-stage detector:

SSD: Single Shot MultiBox Detector

Why Was SSD Developed?

We have already seen two-stage detectors such as Faster R-CNN:

$$\text{Image} \rightarrow \text{Region Proposals} \rightarrow \text{Classification + Box Refinement}$$

This works well, but requires a separate region-proposal stage.

SSD takes a different approach:

Predict object classes and bounding boxes directly from feature maps

There is no separate Region Proposal Network followed by a second classification stage.

Main Goal

Perform object detection in a single detection stage while still handling objects of different sizes and shapes.

What Does SSD Mean?

SSD stands for:

Single Shot MultiBox Detector

Single Shot
Detection is performed directly by one neural-network architecture without a separate region-proposal stage.

MultiBox
The network considers many predefined candidate boxes at many spatial locations and feature-map scales.

Important

"Single Shot" means that object localization and classification are performed within one detection pipeline (One Stage).

Step 1: Extract Image Features

The input image first passes through a CNN backbone.

$$\boxed{\text{Image} \rightarrow \text{Backbone CNN} \rightarrow \text{Feature Maps}}$$

The original SSD paper used a modified **VGG-16** network as its backbone.
Modern SSD implementations may use other backbones, such as MobileNet.

Role of the Backbone

The backbone does not directly output final detections.
It produces learned visual features that the SSD detection heads use to predict objects.

Step 2: Predict at Multiple Feature-Map Scales

A major idea in SSD is that predictions are made from **multiple feature maps**.

Conceptually:

$$\text{Fine Feature Map} \rightarrow \text{More Spatial Locations} \rightarrow \text{Smaller Objects}$$

$$\text{Coarse Feature Map} \rightarrow \text{Fewer Spatial Locations} \rightarrow \text{Larger Objects}$$

Why?

A single feature-map resolution is not equally suitable for objects of every size.
SSD uses several feature-map resolutions in the same network.

Step 3: Default Boxes in SSD

SSD uses predefined **anchor boxes** (called default boxes) at each spatial location of its prediction feature maps.

- Each location has multiple anchors with different scales and aspect ratios.
- Different feature-map resolutions allow SSD to detect objects of different sizes.
- For each anchor, SSD directly predicts object class scores and bounding-box adjustments.

Key Idea

Unlike Faster R-CNN, SSD uses anchors directly for classification and localization, without a separate RPN.

Step 4: SSD Detection Heads

SSD applies small convolutional prediction filters directly to each selected feature map.

At every spatial location, these filters produce two outputs for each associated anchor box:

1. **Class scores:**
$$(p_0, p_1, \ldots, p_K)$$
where p_0 represents background.
2. **Bounding-box offsets:**
$$(\Delta x, \Delta y, \Delta w, \Delta h)$$

Conceptually:

Feature Maps $\rightarrow$ Convolutional Detection Heads $\rightarrow$ Class Scores + Box Offsets

Key Difference from R-CNN

SSD predicts classes and bounding-box adjustments directly from feature maps, without cropping individual proposals or using a separate region-classification stage.

SSD Training Loss

SSD solves two learning problems at the same time:

1. classify each matched default box,
2. localize the object accurately.

The original SSD objective combines:

$$L = \frac{1}{N}\left(L_{\text{confidence}} + \alpha L_{\text{localization}}\right)$$

where:

- $L_{\text{confidence}}$ penalizes incorrect class predictions,
- $L_{\text{localization}}$ penalizes inaccurate box regression,
- N is the number of matched positive default boxes.

SSD During Inference

For a new image:

1. the backbone creates feature maps,
2. default boxes are associated with feature-map locations,
3. SSD predicts class scores and box offsets,
4. offsets are applied to produce candidate boxes,
5. weak predictions are removed,
6. NMS suppresses duplicate detections.

Image $\rightarrow$ Feature Maps $\rightarrow$ Default Boxes $\rightarrow$ Scores + Refined Boxes $\rightarrow$ NMS

SSD vs. Faster R-CNN

Aspect	SSD	Faster R-CNN
Detector type	One-stage	Two-stage
Proposal stage	None	RPN
Candidate boxes	Default boxes	RPN anchors/proposals
Classification	Direct	After proposal stage
Feature scales	Multiple	Architecture dependent
Pipeline	Simpler	More complex

Central Difference

SSD predicts classes and box offsets directly from dense feature maps.
Faster R-CNN first generates proposals and then performs final detection.

SSD Is an Architecture, Not One Pretrained Model

SSD defines the object detection mechanism, while a **CNN backbone** extracts visual features from the image.

The detection pipeline is:

$$\text{Image} \rightarrow \underbrace{\text{MobileNet V2}}_{\text{CNN Feature Extractor}} \rightarrow \text{Feature Maps}$$

$$\text{Feature Maps} \rightarrow \underbrace{\text{SSD Detection Heads}}_{\text{Classification + Box Regression}} \rightarrow \text{Detections}$$

For example, **SSD MobileNet V2 320** combines:

- **SSD:** The object detection architecture.
- **MobileNet V2:** The CNN backbone for feature extraction.
- **320:** The configured input resolution (320×320).

Important

Different SSD models can use different CNN backbones, input resolutions, and pretrained weights.

Pretrained SSD Models in TensorFlow

TensorFlow provides pretrained SSD-family detectors.
Examples include:

- SSD MobileNet V2 320×320,
- SSD MobileNet V1 FPN 640×640,
- SSD MobileNet V2 FPNLite 320×320,
- SSD MobileNet V2 FPNLite 640×640.

Practical Use

A pretrained detector can be used directly for inference or adapted to a custom detection task through transfer learning or fine-tuning.

Using a Pretrained SSD Detector

For inference, we do not manually generate default boxes.
The pretrained detector performs the internal SSD operations for us.

A typical workflow is:

Load Model → Prepare Image → Run Model

Read Boxes, Classes, Scores → Visualize Results

Important

The internal operations we just studied still occur inside the model.
The API simply hides those implementation details from the user.

Current Subsection

1 One-Stage vs. Two-Stage Detectors

2 Feature Maps and Prediction Locations

3 Anchor Boxes

4 One-Stage Anchor-Based Object Detector — SSD Example
- Lab

5 One-Stage Anchor-Free Object Detector — YOLO Example

Loading Image

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

url = 'https://raw.githubusercontent.com/tensorflow/models/master/research/object_detection/test_images/image2.jpg'
path = tf.keras.utils.get_file('ssd_beach.jpg', origin=url)
image = np.array(Image.open(path).convert('RGB'))

plt.figure(figsize=(10, 7))
plt.imshow(image)
plt.axis('off')
plt.show()
```

Result



Loading the Pretrained SSD Model

We import TensorFlow and TensorFlow Hub, then load an SSD MobileNet V2 FPNLite model pretrained on COCO with a 320 × 320 input resolution for object detection.

```
import tensorflow as tf
import tensorflow_hub as hub
import matplotlib.patches as patches

ssd_model = hub.load('https://tfhub.dev/tensorflow/ssd_mobilenet_v2/fpnlite_320x320/1')
```

```
/usr/local/lib/python3.13/dist-packages/tensorflow_hub/__init__.py:61: UserWarning: pkg_resources
  from pkg_resources import parse_version
```

Loading COCO Class Labels

We download the COCO label map and create a dictionary that maps numeric class IDs to object names, allowing SSD predictions to display labels such as person, car, and bicycle instead of numbers.

```
import requests, re

url = "https://raw.githubusercontent.com/tensorflow/models/master/research/object_detection/data/mscoco_label_map.pbtxt"
text = requests.get(url).text
coco_names = {int(i): name for i, name in re.findall(r'id:\s*(\d+)\s+display_name:\s*"([^"]+)"', text)}

print(coco_names)
```

```
{1: 'person', 2: 'bicycle', 3: 'car', 4: 'motorcycle', 5: 'airplane', 6: 'bus', 7: 'train', 8: 'truck', 9: 'boat', 10: 'traffic light',
```

COCO: Common Objects in Context

COCO is a large, labeled image dataset used to train and evaluate object detection and segmentation models.

COCO contains:

- Over 330,000 images, including more than 200,000 labeled images.
- 80 everyday object classes, such as people, cars, dogs, and chairs.
- Ground-truth bounding boxes identifying object locations.
- Segmentation masks identifying which pixels belong to objects.
- Images containing multiple objects at different scales and positions.

Our pretrained model: SSD MobileNet V2 FPNLite (320×320)

$$\underbrace{\text{MobileNet V2}}_{\text{CNN Backbone}} + \underbrace{\text{FPNLite}}_{\text{Multi-scale Features}} + \underbrace{\text{SSD}}_{\text{Detection Head}}$$

Training Dataset: COCO 2017

Predictions

```
predictions = ssd_model(tf.convert_to_tensor(image[None, ...], dtype=tf.uint8))

scores = predictions['detection_scores'][0].numpy()
classes = predictions['detection_classes'][0].numpy().astype(int)

for i in np.argsort(scores)[-10:][::-1]:
    print(f"{coco_names.get(classes[i], classes[i])}: {scores[i]:.3f}")
```

```
kite: 0.452
person: 0.446
person: 0.396
person: 0.330
person: 0.288
person: 0.270
person: 0.267
kite: 0.251
person: 0.247
kite: 0.243
```

Visualize the detected objects

```python
def show_ssd(image, predictions, threshold=0.2):
    boxes = predictions['detection_boxes'][0].numpy()
    scores = predictions['detection_scores'][0].numpy()
    classes = predictions['detection_classes'][0].numpy().astype(int)
    height, width = image.shape[:2]
    fig, ax = plt.subplots(figsize=(10, 7))
    ax.imshow(image)
    count = 0

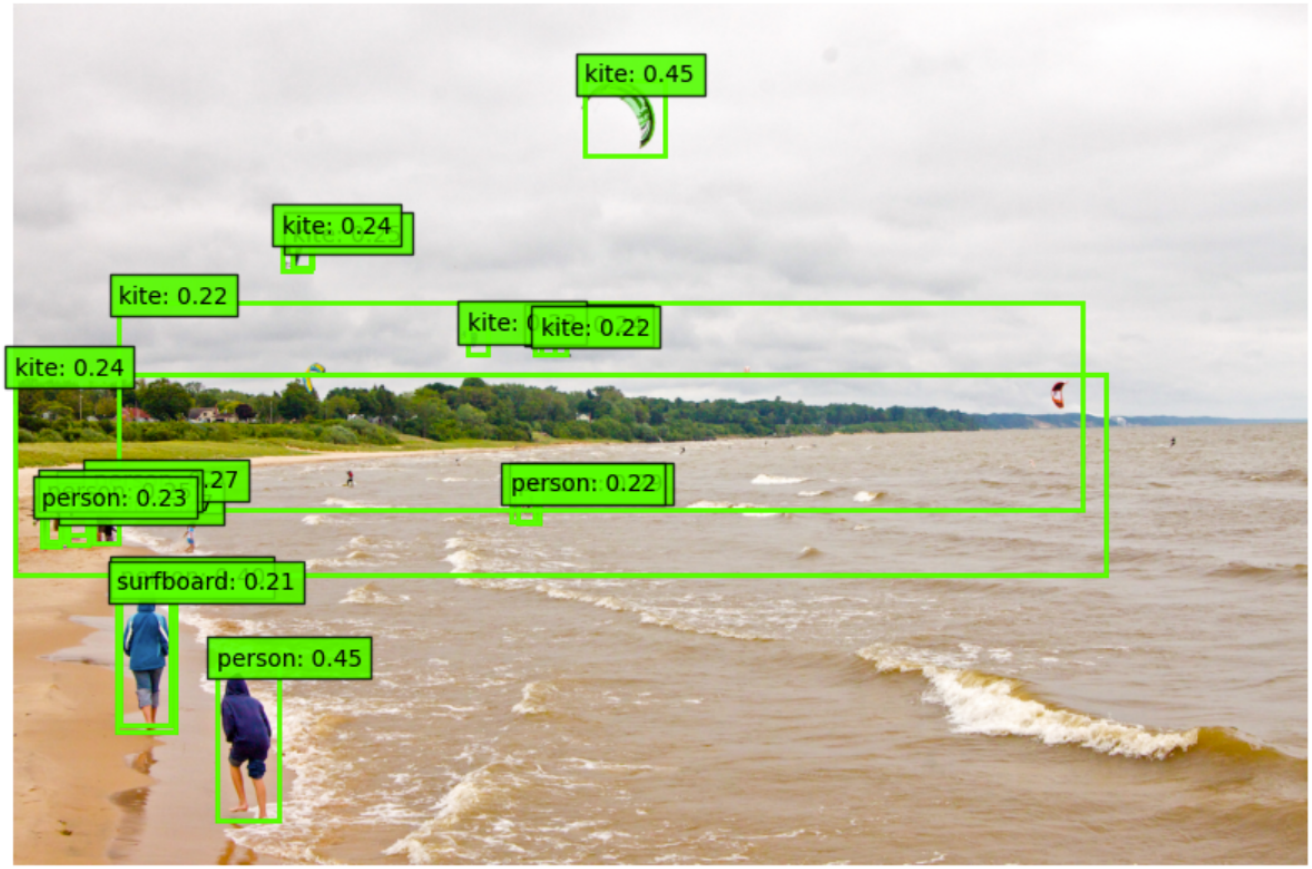
    for box, score, label in zip(boxes, scores, classes):
        if score < threshold: continue
        top, left, bottom, right = box
        x, y = left * width, top * height
        w, h = (right - left) * width, (bottom - top) * height
        ax.add_patch(patches.Rectangle((x, y), w, h, fill=False, edgecolor='lime', linewidth=2))
        ax.text(x, y, f'{coco_names.get(label, label)}: {score:.2f}', color='black', bbox=dict(facecolor='lime', alpha=0.8))
        count += 1

    ax.axis('off')
    plt.show()
    print(f'Objects detected: {count}')

show_ssd(image, predictions, threshold=0.2)
```

Result

...




```
Objects detected: 18
```

Changing the Confidence

```
show_ssd(image, predictions, threshold=0.3)
```




```
Objects detected: 4
```

Current Section

1 One-Stage vs. Two-Stage Detectors

2 Feature Maps and Prediction Locations

3 Anchor Boxes

4 One-Stage Anchor-Based Object Detector — SSD Example

5 One-Stage Anchor-Free Object Detector — YOLO Example
- Original YOLOv1
- Lab

Current Subsection

1 One-Stage vs. Two-Stage Detectors

2 Feature Maps and Prediction Locations

3 Anchor Boxes

4 One-Stage Anchor-Based Object Detector — SSD Example

5 One-Stage Anchor-Free Object Detector — YOLO Example
- Original YOLOv1
- Lab

Why YOLO?

YOLO stands for **You Only Look Once**.

YOLO was designed to make object detection fast by treating detection as a direct prediction problem.

Instead of using a separate region-proposal stage:

$$\boxed{\text{Image} \rightarrow \text{Neural Network} \rightarrow \text{Boxes + Classes + Scores}}$$

Main Idea

YOLO predicts object locations and object classes directly from image features in one detection pipeline.

YOLO as a One-Stage Detector

YOLO is a **one-stage detector**.

It does not first generate region proposals and then classify them.

Instead, the network predicts:

- bounding boxes,
- objectness scores,
- class scores,

directly from feature maps.

$$\boxed{\text{Feature Maps} \rightarrow \text{Detection Head} \rightarrow \text{Candidate Detections}}$$

How Original YOLOv1 Works

Unlike anchor-based detectors, **YOLOv1 does not use predefined anchor boxes**. Instead, it divides the image into an $S \times S$ grid.

- Each grid cell is responsible for an object whose ground-truth center falls inside that cell.
- Each cell predicts B bounding boxes, represented by (x_c, y_c, w, h), along with an objectness confidence for each box (B is the number of bounding boxes each grid cell predicts.).
- The responsible cell learns the object's bounding box using the ground-truth coordinates.
- The predicted box can extend beyond its grid cell. The cell determines **responsibility**, not the boundaries of the predicted box.

$$\boxed{\text{Grid Cell} \rightarrow \text{Box Coordinates + Confidence}}$$

Key Idea

YOLOv1 directly learns object locations and box dimensions without adjusting predefined anchor boxes.

YOLOv1: Objectness and Class Prediction

In addition to bounding boxes, YOLOv1 predicts **confidence scores** and **object class probabilities**.

- **Box confidence:** Estimates object presence and bounding-box overlap with the ground truth.
- **Class probabilities:** Predict which object class is present, such as car, person, or dog.
- **Final class-specific score:** Combines box confidence with the predicted class probability.

For example, if the box confidence is 0.90 and the predicted probability of a car is 0.80:

$$\boxed{\text{Car Detection Score} = 0.90 \times 0.80 = 0.72}$$

Important

YOLOv1 predicts one set of class probabilities per grid cell, but multiple bounding boxes with separate confidence scores.
The final class-specific scores are used to rank detections before filtering redundant boxes.

How Original YOLOv1 Is Trained

During training, each ground-truth object is assigned to the grid cell containing its center.

1. The responsible cell predicts bounding boxes, confidence scores, and class probabilities.
2. The box predictor with the highest IoU is assigned responsibility for the object.
3. Predictions are compared with ground-truth boxes and class labels to calculate the loss.
4. Backpropagation updates the network weights to improve future predictions.

The training loss combines three main components:

$$L_{\text{total}} = \lambda_{\text{box}} L_{\text{box}} + L_{\text{confidence}} + L_{\text{class}}$$

Key Idea

The network learns object locations, bounding-box dimensions, confidence scores, and object classes through a combined loss.

YOLO Inference

For a new input image, YOLO performs object detection in a single stage without a separate region-proposal step.

1. The CNN extracts feature maps from the input image.
2. The detection head predicts candidate bounding boxes and class-specific confidence scores.
3. Predicted coordinates are decoded into bounding boxes.
4. Low-confidence predictions are removed using a confidence threshold.
5. Non-Maximum Suppression (NMS) removes redundant overlapping boxes in YOLO versions that use NMS.

$$\boxed{\text{Image} \rightarrow \text{Feature Extraction} \rightarrow \text{Candidate Detections}}$$

$$\boxed{\text{Confidence Filtering} \rightarrow \text{NMS} \rightarrow \text{Final Detections}}$$

Key Idea

YOLO predicts object classes and bounding boxes directly from image features, without a separate region-proposal stage.

YOLO and Multi-Scale Prediction

Modern YOLO models usually predict at multiple feature-map scales.

For example:

$$80 \times 80, \qquad 40 \times 40, \qquad 20 \times 20$$

Different scales help detect different object sizes.

$$\text{Fine feature map} \rightarrow \text{small objects}$$

$$\text{Coarse feature map} \rightarrow \text{large objects}$$

Connection to SSD

Both SSD and modern YOLO models use dense predictions from feature maps. The exact prediction head and training strategy differ by version.

YOLO Versions: Anchor-Based vs. Anchor-Free

YOLO is a family of one-stage object detectors whose architecture and prediction methods have evolved across versions.

Version	Main Development	Anchor Use
YOLOv1	Grid-based direct prediction	Anchor-free
YOLOv2	Introduced anchor boxes	Anchor-based
YOLOv3	Multi-scale detection	Anchor-based
YOLOv4/v5	Improved architecture and training	Anchor-based
YOLOX	Decoupled detection head	Anchor-free
YOLOv8	Modern anchor-free detection	Anchor-free
YOLO11	Improved efficiency and accuracy	Anchor-free

Key Idea

YOLO refers to a family of one-stage detection architectures, not a single fixed model. Anchor usage, network architecture, and training methods depend on the specific version.

SSD vs. YOLO

Both SSD and YOLO are one-stage detectors.

Aspect	SSD	YOLO
Detector type	One-stage	One-stage
Region proposals	No	No
Prediction style	Dense default-box prediction	Dense YOLO-head prediction
Multi-scale prediction	Yes	Yes in modern versions
Anchor usage	Traditionally yes	Depends on version
Main focus	Simple one-stage detection	Real-time detection family

Main Lesson

Both predict object locations and classes directly from feature maps, without a separate region-proposal stage.

Current Subsection

1 One-Stage vs. Two-Stage Detectors

2 Feature Maps and Prediction Locations

3 Anchor Boxes

4 One-Stage Anchor-Based Object Detector — SSD Example

5 One-Stage Anchor-Free Object Detector — YOLO Example
- Original YOLOv1
- Lab

YOLO11 Lab: Pill Detection and Fine-Tuning

Objective: Fine-tune a pretrained YOLO11 model to detect pharmaceutical pills using transfer learning.

What We Will Do:

1. Load a YOLO11 model pretrained on COCO.
2. Download the Medical Pills dataset (92 training and 23 validation images).
3. Test the pretrained model before fine-tuning.
4. Fine-tune YOLO11 to detect a new object class: **pill**.
5. Compare predictions before and after fine-tuning.
6. Evaluate precision, recall, and mean Average Precision (mAP).
7. Experiment with confidence and IoU thresholds.

Key Idea

Transfer learning allows YOLO11 to reuse pretrained visual features and adapt to a new object detection task using a small labeled dataset.

Imports

We install the Ultralytics library and import the tools needed to load, fine-tune, and evaluate YOLO11 on the Medical Pills dataset.

```
!pip -q install ultralytics
```

```
from ultralytics import YOLO
from ultralytics.data.utils import check_det_dataset
from IPython.display import display
from PIL import Image
from pathlib import Path
import torch
```

Load a pretrained YOLO11 model

```
model = YOLO("yolo11n.pt")
print("First five pretrained classes:", list(model.names.values())[:5])
print("Pill is a COCO class:", "pill" in model.names.values())
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 100% ━━━━━━━━━━━━ 5.4MB 42.4MB/s 0.1s
First five pretrained classes: ['person', 'bicycle', 'car', 'motorcycle', 'airplane']
Pill is a COCO class: False
```

yolo11n.pt starts with COCO-trained weights. n means nano, the smallest model size.

Download the Labeled Pill Dataset

```
dataset = check_det_dataset("medical-pills.yaml")
val_images = sorted(Path(dataset["val"]).glob("*.jpg")) + sorted(Path(dataset["val"]).glob("*.png"))

sample = val_images[0]
print("Class names:", dataset["names"])
print("Validation image:", sample)
display(Image.open(sample).resize((500, 400)))
```

```
WARNING ⚠️ Dataset 'medical-pills.yaml' images not found, missing path '/content/datasets/medical-pills/images/val'
Downloading https://ultralytics.com/assets/medical-pills.zip to '/content/datasets/medical-pills.zip': 100% ———————— 8.2MB 225.7MB/s 0.0s
Unzipping /content/datasets/medical-pills.zip to /content/datasets/medical-pills...: 100% ———————— 238/238 2.0Kfiles/s 0.1s
Dataset download success ✅ (0.8s), saved to /content/datasets

Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100% ———————— 755.1KB 79.4MB/s 0.0s
Class names: {0: 'pill'}
Validation image: /content/datasets/medical-pills/images/val/Frame_144.jpg
```

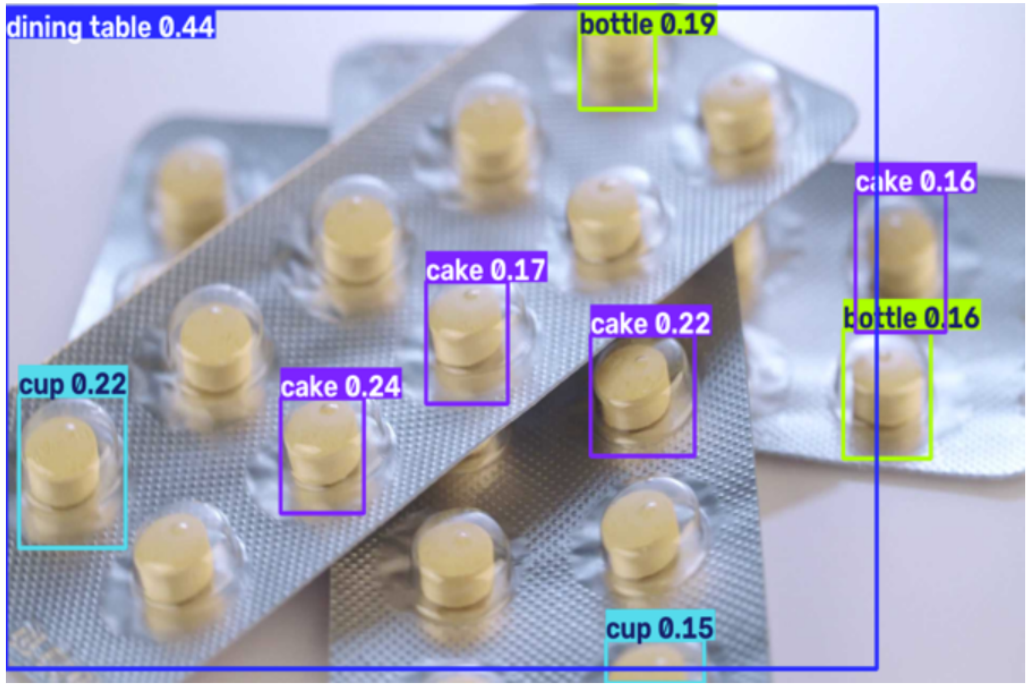


medical-pills.yaml defines the training and validation folders and the single class pill. Ultralytics downloads the dataset automatically. We pick one held-out validation image, not a training image.

Predict BEFORE fine-tuning

A COCO-pretrained detector knows many everyday objects, but it has no pill class.

```
before = model.predict(source=str(sample), conf=0.15, verbose=False)[0]
display(Image.fromarray(before.plot()[..., ::-1]).resize((600, 400)))
print("Detected classes:", [model.names[int(c)] for c in before.boxes.cls.tolist()])
```




```
Detected classes: ['dining table', 'cake', 'cup', 'cake', 'bottle', 'cake', 'cake', 'bottle', 'cup']
```

Fine-tune on the pill dataset

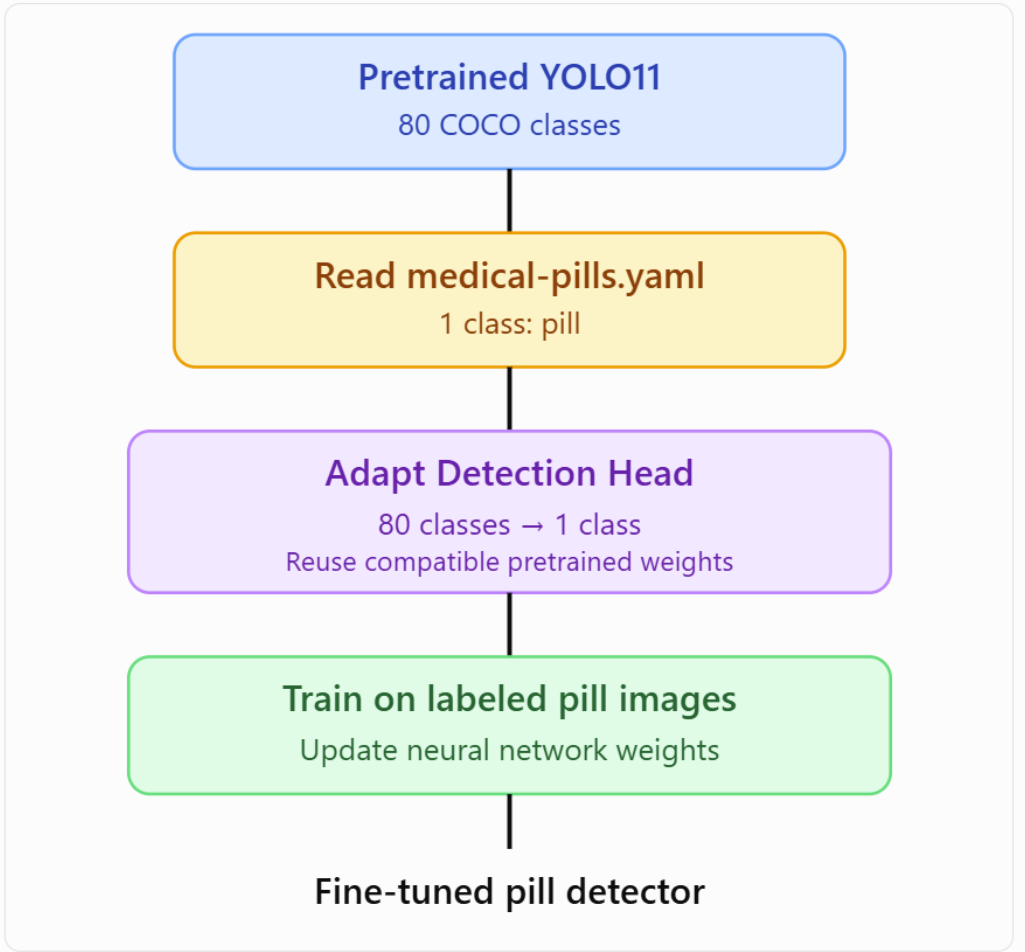
The pretrained feature extractor is reused. The detection head adapts to one class: pill. This cell performs actual training, not just inference.

```
device = 0 if torch.cuda.is_available() else "cpu"
model.train(data="medical-pills.yaml", epochs=12, imgsz=416, batch=8, device=device, project="runs_yolo_lab", name="pills", exist_ok=True)
best_path = model.trainer.best
print("Best checkpoint:", best_path)
```

After fine-tuning YOLO11 on the Medical Pills dataset, the new model can only predict the pill class.

Note: Fine-tuning on a new one-class dataset replaces the original classification task. If you want a single model to detect both the original 80 COCO classes and pills, you need a training setup that includes all 81 classes, with suitable labeled examples to preserve performance on the original classes.

What happens inside model.train()



Result (First Part)

```
Ultralytics 8.4.204 🚀 Python-3.13.18 torch-2.14.1+cpu CPU (Intel Xeon CPU @ 2.20GHz)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=8, bgr=0.0, box=7.5, cache=False,
Overriding model.yaml nc=80 with nc=1

                   from  n    params  module                                       arguments
  0                  -1  1       464  ultralytics.nn.modules.conv.Conv             [3, 16, 3, 2]
  1                  -1  1      4672  ultralytics.nn.modules.conv.Conv             [16, 32, 3, 2]
  2                  -1  1      6640  ultralytics.nn.modules.block.C3k2            [32, 64, 1, False, 0.25]
  3                  -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
  4                  -1  1     26080  ultralytics.nn.modules.block.C3k2            [64, 128, 1, False, 0.25]
  5                  -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
  6                  -1  1     87040  ultralytics.nn.modules.block.C3k2            [128, 128, 1, True]
  7                  -1  1    295424  ultralytics.nn.modules.conv.Conv             [128, 256, 3, 2]
  8                  -1  1    346112  ultralytics.nn.modules.block.C3k2            [256, 256, 1, True]
  9                  -1  1    164608  ultralytics.nn.modules.block.SPPF            [256, 256, 5]
 10                  -1  1    249728  ultralytics.nn.modules.block.C2PSA           [256, 256, 1]
 11                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 12             [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 13                  -1  1    111296  ultralytics.nn.modules.block.C3k2            [384, 128, 1, False]
 14                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 15             [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 16                  -1  1     32096  ultralytics.nn.modules.block.C3k2            [256, 64, 1, False]
 17                  -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
 18            [-1, 13]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 19                  -1  1     86720  ultralytics.nn.modules.block.C3k2            [192, 128, 1, False]
 20                  -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
 21            [-1, 10]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 22                  -1  1    378880  ultralytics.nn.modules.block.C3k2            [384, 256, 1, True]
 23        [16, 19, 22]  1    430867  ultralytics.nn.modules.head.Detect           [1, 16, None, [64, 128, 256]]
YOLO11n summary: 181 layers, 2,590,035 parameters, 2,590,019 gradients, 6.5 GFLOPs
```

YOLO11: Model Architecture and Fine-Tuning

Key Information from the Training Output:

- **Class adaptation (Overriding model.yaml nc=80 with nc=1):** The detection head changes from 80 COCO classes to 1 class (`pill`).
- **Architecture:** YOLO11n contains 181 layers and approximately 2.59 million parameters.
- **Feature extraction:** Convolutional blocks and attention modules extract visual features.
- **Attention mechanism (ultralytics.nn.modules.block.C2PSA):** Layer 10 contains the C2PSA (Cross-Stage Partial with Position-Sensitive Attention) module, which helps the model capture spatial relationships and refine visual features.

Summary

YOLO11 reuses pretrained visual features, adapts its detection head to predict the new `pill` class, and updates its weights through backpropagation during fine-tuning.

Results (Last Part)

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      11/12         0G     0.9545     0.5636     0.9731         59        416: 100% ━━━━━━━━━━━━ 12/12 2.2s/it 26.3s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.2it/s 1.7s
                   all         23        399      0.985      0.972      0.992      0.732

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      12/12         0G     0.9493       0.55     0.9584         76        416: 100% ━━━━━━━━━━━━ 12/12 2.2s/it 26.1s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.1it/s 1.8s
                   all         23        399      0.986      0.972      0.992      0.738

12 epochs completed in 0.098 hours.
Optimizer stripped from /content/runs/detect/runs_yolo_lab/pills/weights/last.pt, 5.5MB
Optimizer stripped from /content/runs/detect/runs_yolo_lab/pills/weights/best.pt, 5.5MB

Validating /content/runs/detect/runs_yolo_lab/pills/weights/best.pt...
Ultralytics 8.4.204 🚀 Python-3.13.18 torch-2.14.1+cpu CPU (Intel Xeon CPU @ 2.20GHz)
YOLO11n summary (fused): 101 layers, 2,597,051 parameters, 0 gradients, 6.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.3it/s 1.6s
                   all         23        399      0.986      0.972      0.992      0.738
Speed: 0.5ms preprocess, 44.8ms inference, 0.0ms loss, 8.5ms postprocess per image
Results saved to /content/runs/detect/runs_yolo_lab/pills
Best checkpoint: /content/runs/detect/runs_yolo_lab/pills/weights/best.pt
```

YOLO11 Fine-Tuning: Training Results

Training Configuration:

- 12 epochs, 416×416 input resolution.
- 92 training images and 23 validation images.
- 399 annotated pill instances in the validation set.

Final Validation Performance:

- **Precision:** 98.6%
- **Recall:** 97.2%
- **mAP@50:** 99.2%
- **mAP@50:95:** 73.8%

Key Findings

YOLO11 successfully learns to detect pills using transfer learning. The model achieves high detection precision and recall, while the lower mAP@50:95 indicates room for improvement in bounding-box localization.

YOLO11: Confidence, Precision, and Recall

Confidence Score: How confident the model is about an individual detection. For example, `pill: 0.95` indicates a detection score of 95%.

Precision: Of all predicted pills, how many are correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: Of all actual pills, how many did the model detect?

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **TP (True Positive):** A correctly detected pill.
- **FP (False Positive):** An incorrect pill detection.
- **FN (False Negative):** A pill the model missed.

Key Difference

Confidence measures the score of one prediction, while **precision** and **recall** evaluate performance across multiple predictions using ground-truth annotations.
Increasing the confidence threshold generally increases precision but may reduce recall.

YOLO11 Evaluation: Understanding mAP

Mean Average Precision (mAP) measures object detection performance by combining precision and recall across confidence thresholds.

Intersection over Union (IoU):

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

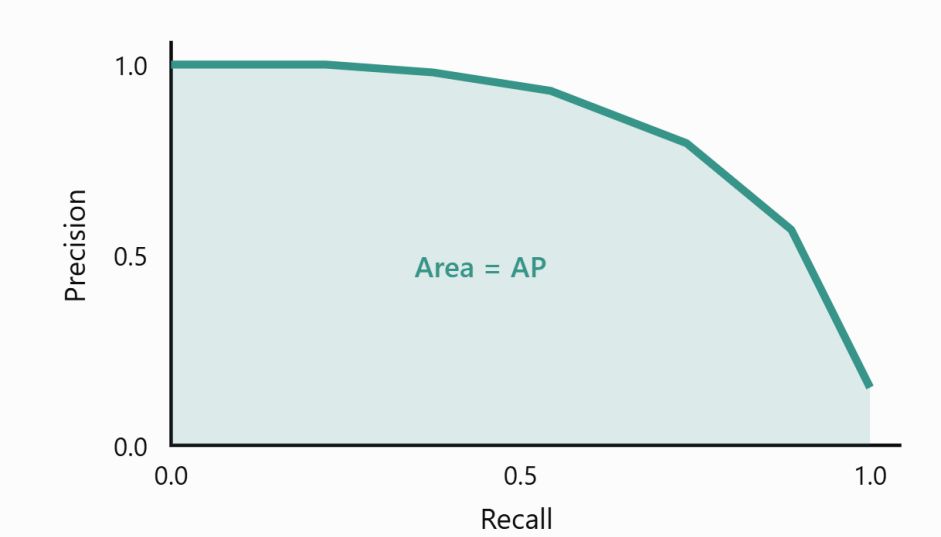
- **mAP@50 = 99.2%:** Average Precision at an IoU threshold of 0.50. A predicted box must overlap the ground-truth box by at least 50% IoU to count as correctly localized.
- **mAP@50:95 = 73.8%:** Average Precision across IoU thresholds from 0.50 to 0.95, in increments of 0.05. This evaluates detection under increasingly strict localization requirements.

Key Takeaway

YOLO11 detects pills very well at IoU = 0.50 (99.2% AP), but its lower mAP@50:95 (73.8%) indicates that bounding-box localization is less accurate under stricter overlap requirements.

YOLO11: Understanding Average Precision (AP) and mAP

Average Precision (AP) summarizes the area under the precision-recall curve as the confidence threshold changes. And **mAP:** Mean of AP values across object classes.



Key Takeaway

Lowering the confidence threshold generally increases recall but may reduce precision. AP summarizes this trade-off across confidence thresholds. Since our dataset has only one class (`pill`), mAP equals AP at each IoU threshold.

Evaluation Code

```python
metrics = model.val(data="medical-pills.yaml", imgsz=416, device=device)
print(f"Precision: {metrics.box.mp:.3f}")
print(f"Recall:    {metrics.box.mr:.3f}")
print(f"mAP50:     {metrics.box.map50:.3f}")
print(f"mAP50-95:  {metrics.box.map:.3f}")
```

```
Ultralytics 8.4.204 🚀 Python-3.13.18 torch-2.14.1+cpu CPU (Intel Xeon CPU @ 2.20GHz)
YOLO11n summary (fused): 101 layers, 2,597,051 parameters, 0 gradients, 6.4 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 1274.1±326.5 MB/s, size: 82.8 KB)
val: Scanning /content/datasets/medical-pills/labels/val.cache... 23 images, 0 backgrounds
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                   all         23        399      0.986      0.972      0.992      0.738
Speed: 0.9ms preprocess, 76.2ms inference, 0.0ms loss, 8.7ms postprocess per image
Results saved to /content/runs/detect/val-2
Precision: 0.986
Recall:    0.972
mAP50:     0.992
mAP50-95:  0.738
```

Visualizing Loss

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("runs/detect/runs_yolo_lab/pills/results.csv")
df.columns = df.columns.str.strip()

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, loss in zip(axes, ["box_loss", "cls_loss", "dfl_loss"]):
    ax.plot(df["epoch"], df[f"train/{loss}"], label="Training", marker="o")
    ax.plot(df["epoch"], df[f"val/{loss}"], label="Validation", marker="o")
    ax.set_title(loss)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Loss")
    ax.legend()
    ax.grid(True)

plt.tight_layout()
plt.show()
```

YOLO11: Understanding Training and Validation Losses

YOLO11 uses three loss components to learn object classification and bounding-box localization.

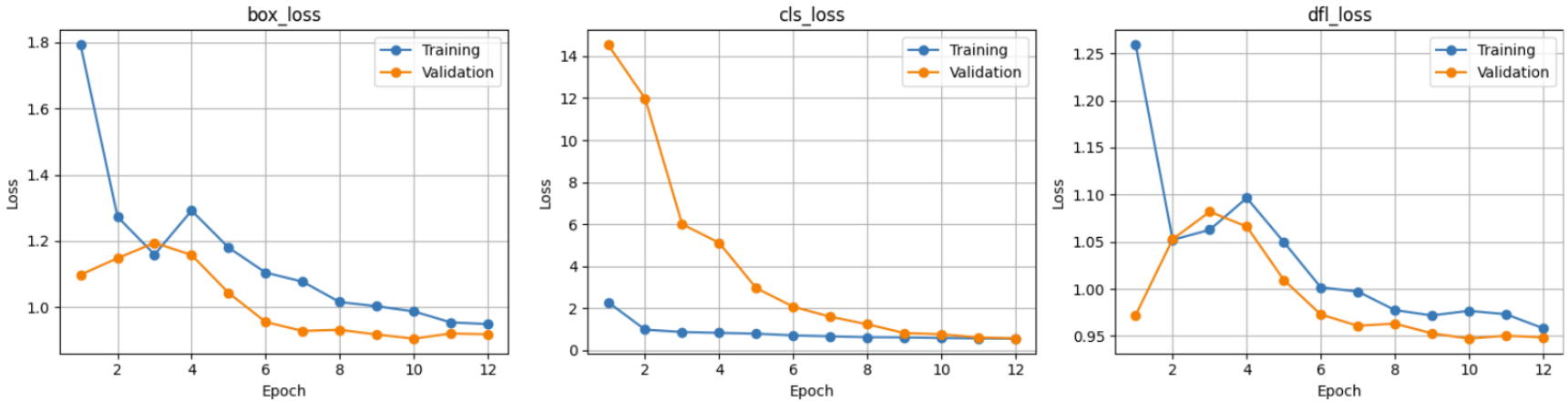
- **Box Loss** (`box_loss`)**:** Measures how accurately predicted bounding boxes match the ground-truth boxes. It uses an IoU-based regression loss. Lower values indicate better localization.

- **Classification Loss** (`cls_loss`)**:** Measures errors in predicted object class scores. In our dataset, the model learns to recognize the `pill` class. Lower values indicate better classification predictions.

- **Distribution Focal Loss** (`dfl_loss`)**:** Measures errors in the predicted distributions of distances to bounding-box edges, helping the model estimate box coordinates more precisely.

  Focal Loss is a loss function that makes a neural network focus more on difficult predictions and less on easy ones. For example, when detecting pills, the model gives less weight to a pill correctly identified with 99

Result

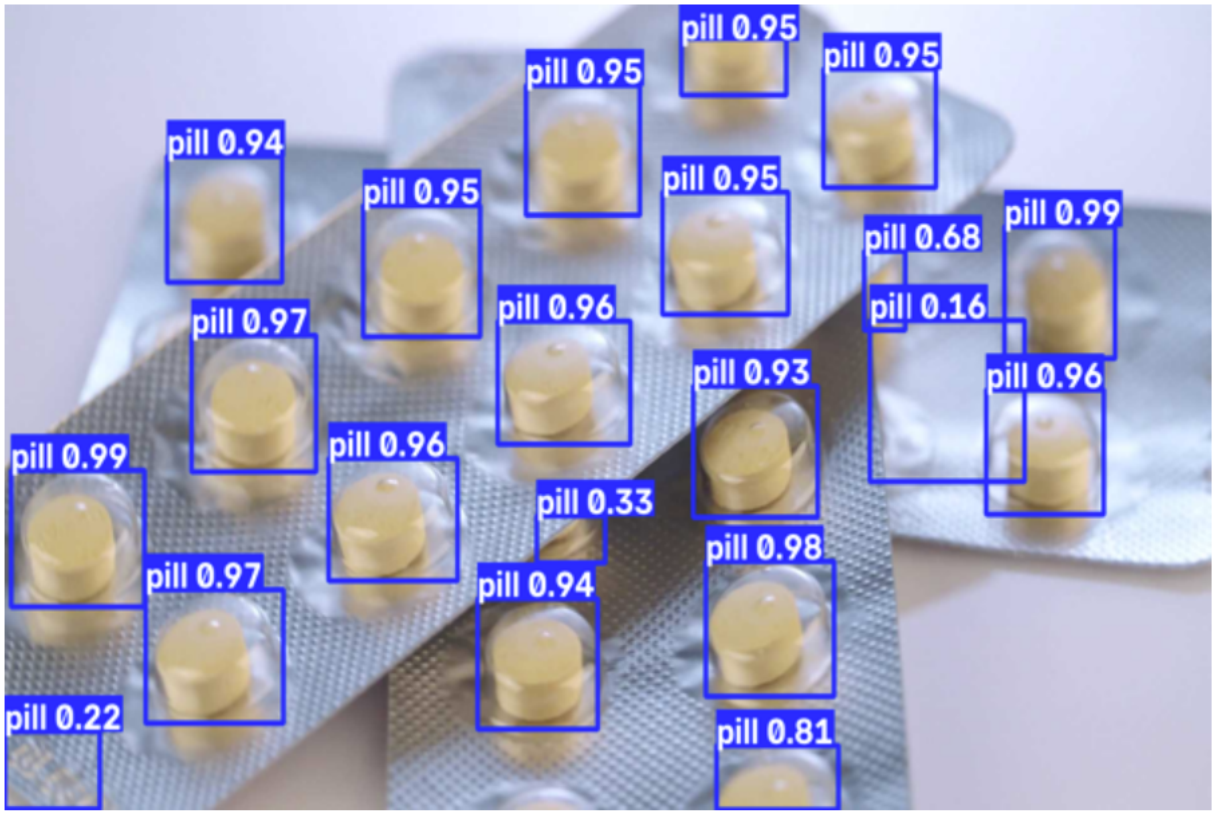


YOLO11 demonstrates successful fine-tuning, as training and validation losses generally decrease across all three components, indicating improved pill classification and bounding-box localization without clear evidence of overfitting.

Load the fine-tuned weights and predict AFTER training

We use exactly the same validation image and confidence threshold as before.

```
fine_tuned = YOLO(str(best_path))
after = fine_tuned.predict(source=str(sample), conf=0.15, verbose=False)[0]
display(Image.fromarray(after.plot()[..., ::-1]).resize((600, 400)))
print("Pills detected:", len(after.boxes))
```

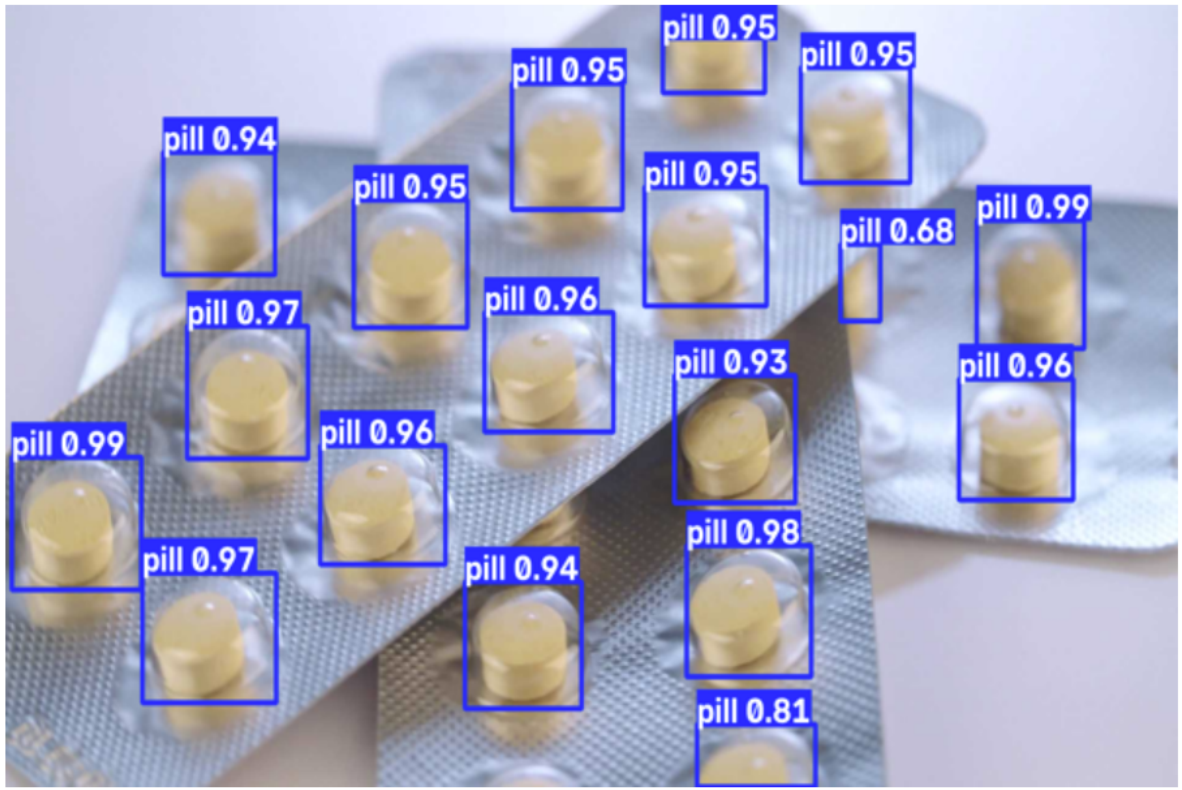



```
Pills detected: 21
```

Experiment with confidence and IoU thresholds

Here conf filters low-scoring predictions. iou controls NMS overlap suppression.

```
result = fine_tuned.predict(source=str(sample), conf=0.35, iou=0.5, verbose=False)[0]
display(Image.fromarray(result.plot()[..., ::-1]).resize((600, 400)))
print("Pills shown:", len(result.boxes))
```




```
Pills shown: 18
```

Congratulations!

Now you know about famous models like YOLO!

